Contents

Proposal: Scheduled Backup Disk

1. What and why
2. Affected domains
3. Design
4. Blank stick → Backup Disk
5. Console UX
6. Tests
7. Rollout
8. Options considered
9. Settled by Steve (no decision needed)
10. Questions for Koen (all answered)

Proposal: Scheduled Backup Disk

Author: Steve (Lead Bot) **Date:** 2026-10-30 **Revised:** 2026-10-30 (Kid's App review, Pixel's Console review and Atlas's rollout review applied, accepted by Steve); 2026-10-30 (Axle's Engine review: 05:00 reboot wrapper, ENGINE_CAPABILITIES, minimum helper version; design review complete) **Status:** Accepted by Koen on 2026-10-30 18:17 CET. Previously: Draft for Koen. Koen chose Option A on 2026-10-29 11:45 CET. Design Review input from Axle (idea#78 comment 6518163224) and the review chat (Axle, Atlas, Kid, Pixel) is folded in. **Refs:** idea#78. Related: idea#235/#236 (erase guards), #279/#282 (adopt, system-disk refusal), #347 (helper rollout order), `proposals/files-disk.md` (eraseDisk). **Affects:** `agent-engine-dev` (main work), `agent-console-dev`, the App repos (`app-kolibri`, `app-nextcloud`, `app-kiwix`, `app-milkwise`) and `agent-app-dev` conventions, Ops (root helper v6 rollout, hardware tests).

---

1. What and why

A Backup Disk today only backs up when it is plugged in (`immediate`) or when someone asks (`on-demand`). The `scheduled` mode is accepted but never runs. There is no cleanup, so the disk fills up. Every backup stops the App. And there is no clear path from a blank USB stick to a Backup Disk.

This proposal makes a Backup Disk that stays plugged in and:

1. backs up every App instance on the Pi **once a day** at a set local time (default 02:30);
2. **clears old backups automatically when the disk gets full**, always keeping the newest few;
3. backs up Apps **without stopping them** when the App says how to do that safely, and stops them briefly when it doesn't;
4. can be **made from a blank stick in one Console action**.

The first user is MilkWise production on `idea` (wh), which today runs an interim cron copy (see idea#78). That interim job is removed only after this ships, by Atlas or Koen, never with a container restart.

2. Affected domains

Domain	Work
Engine	Scheduler tick, schedule in `BACKUP.yaml`, clock trust, fullness cleanup, per-App live backup with hooks, `makeBackupDisk` op, store summary, capability flag.

Domain	Work
Root helper (Ops)	v6 adds `borg-prune`, `borg-delete`, `borg-compact`. Atlas signs off and rolls out before the Engine build.
Console	Schedule settings on the disk, status per App, grey "time unknown" state, disk-full warning, "Make Backup Disk" action.
App repos	`x-app.backup` in each `compose.yaml` (one method, pre/post hooks naming their service). Kolibri, Nextcloud, Kiwix, MilkWise each get theirs.
Ops	Hardware tests on idea04, rollout order, removal of the MilkWise interim job later.

3. Design

3.1 Where the schedule lives

- The schedule belongs to the **Backup Disk**, written in `BACKUP.yaml` on the disk: `schedule: { daily: "02:30" }`. The disk is the source of truth. The Engine the disk is docked to mirrors it into the store (`backupConfig`), as it does for `mode` today.
- Times are **local to the Pi**. A disk moved to another Pi runs at 02:30 on its new Pi.
- Only "daily at HH:MM" in v1. No cron syntax.
- **Only the Engine the disk is docked to runs it** (`dockedTo === localEngine`). No coordination across Engines, so no double runs.
- The scheduler runs **inside the Engine under pm2**, as a one-minute tick next to `src/monitors/timeMonitor.ts`. Never a host cron.
- A scheduled run backs up only instances whose App Disk is docked on the same Engine. Otherwise it records "App Disk not docked" for that instance. No backups over the network in v1.
- Two new capabilities go into `ENGINE_CAPABILITIES` in `src/data/Engine.ts` (the idea#128 list, rewritten at every boot with `capabilitiesBootedAt`):
- `scheduledBackups`: this Engine runs the schedule. The Console only offers scheduling to Engines that report it.
- `backupCleanup`: advertised **only when root helper v6 or newer is present**, the same pattern as `flashCard` and its helper. The Console reads it to show the HELPER_OUTDATED state.

3.2 When a run is due, and the clock

The Pi has no real-time clock, so after an offline boot the time can be wrong.

- `BACKUP.yaml` keeps `lastRun` (time and Engine uptime at that moment) and `lastAttempt`, `missedRuns`.
- **Clock trust.** The Engine computes `clockTrusted` and `clockReason`. The clock is untrusted when it:
- was never synced since boot (by NTP or by a peer Engine), or
- is earlier than the Engine build timestamp, or
- is earlier than the newest timestamp in the store, or
- is more than a day ahead of a synced peer.
- **Trusted clock:** a run is due when local time has passed today's slot and `lastRun` is before that slot. After a missed day (disk docked late, Engine down) the overdue run happens **once**, not once per missed day.
- **Untrusted clock never stops backups.** The Engine falls back to uptime: one backup per 24 hours of uptime since the last run recorded in `BACKUP.yaml`. A run never depends on the wall-clock date.
- A `lastRun` that lies in the future is treated as stale. A backwards clock jump never causes a second run and never causes a prune.
- `checkedAt` and all result times are set by the Engine.

3.3 Locks and interruptions

- Reuse today's locks: the instance lock plus the Backup Disk lock (`backupLockKeysFor`) and `runningBackupOnDisk` (both in `src/monitors/backupMonitor.ts`). A scheduled run that finds a lock held **skips and records the skip**. It never queues or waits.
- **Eject and unplug always win.** Eject keeps today's in-flight rules. A USB pull leaves `.backup-in-progress`; on the next dock the Engine retries **once**, then waits for the next slot.
- **The 05:00 reboot.** The Pis do a full host reboot at 05:00 from root's crontab (`0 5 * * * /sbin/shutdown -r now` in `script/build_image_assets/crondefs`). It is not a pm2 restart, and the Engine can't hold it. The fix:
- Replace that cron line with a small **reboot wrapper script** that waits while `/run/idea/backup-running` exists, up to a **90-minute cap**, then reboots anyway. Steve chose 90 minutes: a daily run with a hook normally takes well under an hour, and the cap still gets the reboot done before school starts.
- The Engine writes the marker when a run starts and removes it when the run ends (including on failure). `/run` is cleared at boot, so a stale marker can't block the next day.
- This is an image/`boot.sh` change that Atlas rolls out, carried with engine#292.
- Kits and any Engine still on the old crontab will still reboot in the middle of a run. They rely on INTERRUPTED plus a `borg check` at the start of the next run.
- Anything interrupted is recorded as **INTERRUPTED** and never counts as the newest good backup (its partial borg checkpoint is not used for the "keep newest N" rule).

- An App that was Running before the backup is always Running afterwards, whatever happens.

3.4 Cleanup driven by how full the disk is

Koen's direction: when the disk fills up, old backups are cleared automatically.

- After each successful backup, and before a backup if space looks tight: if free space is **below 15 %**, delete the oldest backups until free space is **above 25 %**, then `borg compact` to give the space back.
- **Always keep the newest 3 good backups per instance**, and never delete an instance's only backup.
- Several instances on one disk: delete round-robin across instances, oldest first, never touching any instance's newest 3.
- Before a backup, estimate the needed space from the last backup's added size. If it doesn't fit even after cleanup, refuse that backup with **BACKUP_DISK_FULL**. The Console shows "Backup Disk full: add a bigger disk". Nothing is ever deleted down to zero.
- Cleanup uses **fullness and backup count only**, never age, so a wrong clock can't delete good backups.
- Cleanup must work at 15 % free (borg needs some working space; tested explicitly).
- **Root helper v6** adds `borg-prune`, `borg-delete` and `borg-compact` to `idea-app-data` (helper v5 today). Today the Engine checks for exactly `idea-app-data 5` (`APP_DATA_HELPER_VERSION` in `src/utils/appDataHelper.ts`). That becomes a **minimum-version check**: v5 or newer for backups, v6 or newer for cleanup. A v5 helper then gives **HELPER_OUTDATED** (cleanup skipped, backup still runs) instead of a failed helper check. Check the borg version on the image (compact needs 1.2+).

3.5 Consistency: back up without stopping, or stop briefly

Koen's concern: copying files while an App writes its database gives a broken copy. The rule is: **a live copy is only taken when the App declares how it is safe; otherwise the App is stopped.**

Each App declares this in its `compose.yaml` as `x-app.backup`, next to the other `x-app` fields. The schema allows **one method per App**:

```
x-app:
  backup:
    method: hook        # none | live | hook | stop (default: stop)
    preHook:
      service: db       # the compose service the command runs in
      command: "<command>"
      user: "<uid>"     # optional
    postHook:           # optional; always runs, even after a failed preHook
```

```
    service: app
    command: "<command>"
  hookTimeout: 300    # seconds
  exclude: ["<path>"] # optional, never copied
```

- `stop` (default): today's behaviour: stop, borg create, start.
- `live`: copy while running. Only for Apps that write every file via temp file + rename and have no database.
- `hook`: the preHook writes a consistent dump into the App's data folder (under `./data/`), borg copies the folder including the dump, and restore uses the dump.
- `none`: nothing on the Backup Disk for this App, because its content is on the App Disk and can be rebuilt.
- Hooks run with `docker exec` in the **named service's container**, with a timeout, while **holding the instance lock throughout**. The dump is cleaned up afterwards. No root helper change.
- **The postHook always runs**, even when the preHook failed or timed out, so anything the preHook switched on (for example maintenance mode) is always switched off again.
- **If a preHook fails or times out, the Engine falls back to stop-the-App** for that run, never to a plain live copy. This is recorded as **HOOK_FALLBACK** in the per-App result. `method: none` is also recorded.
- No filesystem snapshots in v1: ext4 on USB has none, and LVM/btrfs would change the disk format and identity paths.
- A night stop/start is safe: all Apps use `pull_policy: never`, so a restart can't pull images.
- **A restore keeps the instance's** `.env` (`port_zip`, `MORANGO_SYSTEM_ID`), so a restored Kolibri keeps its kit ID and every App keeps its port.

Per App:

App	Method	How
MilkWise	`live`	Believed to write every file via temp file + rename with no database. **This is not verified yet.** It is checked by a hook/live test on a scratch MilkWise instance on a pool Pi, never on wh. If it doesn't hold, MilkWise gets a hook instead. Any change to `app-milkwise` needs Koen's approval.
Kolibri (0.19.5)	`hook`	python3 inside the container uses SQLite's online backup API to snapshot **every** Kolibri SQLite database (`db.sqlite3`, `job_storage` and the notification databases, all in WAL mode) into `./data/backup/`. Channel content can change during an import, so while a channel import job is active the run falls back to `stop` for that night. The import temp folders are always excluded.

App	Method	How
Nextcloud + MariaDB	`hook`	preHook: `occ maintenance:mode --on` in the `app` service as uid 33, then `mariadb-dump --single-transaction` in the `db` service into `./data/backup/` so borg includes it (MariaDB 11.7 no longer ships the `mysql*` command names). postHook: `occ maintenance:mode --off` in `app` as uid 33, which always runs. User files are copied live; a file being uploaded at 02:30 may be missed and is picked up the next night.
Kiwix	`none`	The ZIMs are on the App Disk and the library file is rebuilt from them, so nothing is backed up.

Kid defines the `x-app.backup` contract in `agent-app-dev` (and the x-app checker) and adds the blocks to each App repo, with a Pi test of each hook.

3.6 What the store holds

- Only a **summary**: per App instance the latest result (time, method, result code such as OK, SKIPPED_LOCKED, HOOK_FALLBACK, INTERRUPTED, BACKUP_DISK_FULL, HELPER_OUTDATED) and a short history (last 7), plus disk `freePct`, `clockTrusted`, `clockReason`, `missedRuns`, and `cleaning` (boolean, with its start time) while cleanup runs.
- Written **once per run**, never per tick, so the store doesn't grow.
- The full list of backups stays on the disk (borg).

4. Blank stick → Backup Disk

One Engine op, `makeBackupDisk`, with steps and result codes like `moveApp`:

1. **Check:** refuse the system disk and any disk the existing erase guards refuse (idea#235/#236, #282 fix). Show the content summary.
2. **Erase:** reuses the `eraseDisk` path and its `isSystemDiskRecord` refusal (`src/data/Commands.ts:475`) (GPT, one ext4 partition, `META.yaml`, label "IDEA Disk"). Typed confirmation in the Console, as for Erase.
3. **Re-dock:** the Engine waits for the disk to re-appear after formatting and adopts it **exactly once**.
4. **Create:** writes `BACKUP.yaml` with mode `scheduled`, the time (default 02:30), and initialises the borg repo.

An empty IDEA disk or an existing App/Files Disk skips step 2 (combined disks are allowed). Afterwards the disk holds `META.yaml`, `BACKUP.yaml` and a borg repository folder. No new sudoers entry and no new root path: erase uses the existing erase script.

5. Console UX

- Disk card: **Make Backup Disk** (with erase summary and typed confirmation when needed).
- Backup Disk settings: mode (immediate / on-demand / scheduled) and time.
- Status per App: last good backup, method, result. Green when fine, red when overdue or failed, **grey "Backup time unknown: this Pi's clock may be wrong"** when `clockTrusted` is false (instead of a red overdue warning).
- Per App, from the result codes in §3.3–3.5: **HOOK_FALLBACK** shows amber "Backed up, but the App was briefly stopped". `method: none` shows neutral, never red: "Not backed up: its content is on the App Disk".
- Disk: free space; amber "Cleaning up old backups" (from `cleaning` in the store), red "Backup Disk full".
- "Cleanup paused: Pi needs an update" for HELPER_OUTDATED.

6. Tests

- Unit tests with a fake clock and `skipBorg` (existing test switch): due logic, missed run runs once, uptime fallback, future `lastRun`, backwards jump, lock skip, retry-once, cleanup choice, round-robin, keep-newest-3, disk-full.
- Wrong-clock tests on a Pi use a **test-only Engine time-offset setting** (used by the scheduler and the clock-trust check), never the system clock, because idea04 is on the shared pool store.
- On idea04 with real borg: prune/delete/compact through helper v6; cleanup at 15 % free; USB pull mid-create, mid-prune and mid-compact, then `borg check`; eject during a run before and after the App stop; two instances on one disk; the reboot wrapper waits while `/run/idea/backup-running` exists, reboots at the 90-minute cap, and is not blocked by a stale marker after boot; on an Engine with the old crontab, a run cut by the 05:00 reboot is marked INTERRUPTED and the next run starts with a `borg check`.
- `makeBackupDisk` on a Pi with an **unplug/replug round trip**; re-dock after formatting happens exactly once; system-disk refusal.
- Per-App hooks: Kolibri (including a run during a channel import, which must fall back to `stop`), Nextcloud (including a failed dump, after which maintenance mode must still be off) and MilkWise backup while in use, then restore and check the data. The MilkWise test runs only on a scratch instance on a pool Pi, never on wh.
- **Every App's restore test checks that `.env` is kept** (`port_zip`, `MORANGO_SYSTEM_ID`); for Kolibri, the restored instance keeps its kit ID.
- Pool rules apply (75/65 °C).

7. Rollout

Precondition: borg 1.2 or later on the image (needed for `compact`). Atlas checks this on idea04 during the proof run.

Engine PRs in this order:

1. Schedule, tick, clock trust, missed runs, store summary, capability (no helper change).
2. Helper v6 and fullness cleanup.
3. Live backup with App hooks, together with Kid's `x-app.backup` contract and the App PRs.
4. `makeBackupDisk` and the Console work (Pixel).

Helper v6 rollout (Atlas):

1. Test helper v6 from the PR branch on a claimed idea04 (proof run, including the borg version check).
2. Koen merges.
3. Install helper v6 on golden idea01, then on idea03, **one Pi at a time**.
4. On each Pi the helper is installed **before** the new Engine.
5. Golden only ever gets merged main.
6. **05:00 reboot wrapper:** Atlas rolls out the image/`boot.sh` change with engine#292, on the same Pis in the same order. Pis without it (kits, wh) still reboot at 05:00 even mid-run and rely on INTERRUPTED plus a `borg check` on the next run.
7. **Deployed Engines (Koen, 30 Oct 18:19 CET):** the kits and wh don't get the new backup feature (Engine or helper v6) until their planned update (see idea#374). They then get both together, so the "helper v5, no cleanup" (HELPER_OUTDATED) case doesn't happen on deployed Engines. HELPER_OUTDATED stays only as a safety check.

MilkWise on wh is part of wh's planned update (idea#374): the `MILKWISE-BAK` conversion happens then, using the plain file copy method in §10 Q2. The interim cron job is removed only after the automated restore check passes, without restarting the container.

8. Options considered

- **Host cron (like the interim job):** ruled out. Outside the Engine, invisible to the Console, not tied to the disk.
- **Schedule in the synced store:** every Engine would see it and could double-fire; doesn't work when the disk moves offline. Ruled out.
- **Cron syntax:** more parsing and testing; nobody needs it. Daily only.
- **Age-based retention (keep 30 days):** depends on a correct clock and doesn't protect against a full disk. Koen asked for fullness. Ruled out.

- **Filesystem snapshots (LVM/btrfs):** would change the disk format and identity code. Ruled out for v1.
- **Always live copy:** risks broken database copies (Koen's concern). Ruled out; live only when declared safe.
- **Always stop the App:** not acceptable for MilkWise production. Kept only as default and fallback.
- **Wait for a synced clock before backing up:** a school Pi offline for weeks would never back up. Replaced by the uptime fallback.
- **Blank stick as three separate Console steps:** more room for mistakes; one op with refusable steps reuses the same safe paths.

9. Settled by Steve (no decision needed)

Default time 02:30; cleanup thresholds 15 % / 25 %; keep newest 3; daily only; history of 7 in the store; hook timeout 300 s; retry once after an interrupted run; per-App methods in §3.5; rollout order in §7.

10. Questions for Koen (all answered)

Koen accepted this proposal on 30 Oct 2026, 18:17 CET.

1. **Nextcloud:** is a short maintenance-mode window (around a minute at 02:30) acceptable, or must Nextcloud never pause? **Answer (30 Oct 18:17 CET):** Yes. Koen accepts a short Nextcloud maintenance-mode window of about a minute at 02:30.
2. **MilkWise on wh:** may we make a proper Backup Disk for it (new stick or reuse the interim `MILKWISE-BAK` stick, which would be erased) and then remove the interim cron job? **Answer (30 Oct 18:17 CET):** Koen agrees to turn the interim `MILKWISE-BAK` flash stick into a proper Backup Disk (it gets erased). This is a physical step for Koen on idea#375. **MilkWise method (MilkWise Dev bot, 30 Oct):** a plain file copy of the whole MilkWise data folder (`/app/data` in the container): no database, no dump hook, no pause. The JSON files are written via temp file and rename, the JSONL history is append-only, and readers skip a partial last line; temp files may be skipped. Never stop, remove or recreate the MilkWise container or its volume. The interim cron job is removed only after an automated restore of the data folder to a scratch location passes and the restored JSON parses. `MILKWISE-BAK` 's last copy is kept until then, and the stick is erased only after that. Kid still confirms the exact host path mounted at `/app/data` from `app-milkwise` . Nothing changes on wh without Koen's own go-ahead.
3. **Kits and wh on helper v5:** OK that they back up without automatic cleanup until a helper update is rolled out there? **Answer (Koen, 30 Oct 18:19 CET):** "Current deployed engines only get the new backup as part of a scheduled update procedure." The kits and wh get the Engine change and helper v6 together in their planned update (idea#374), so the no-cleanup case doesn't happen on deployed Engines. See §7 step 7.